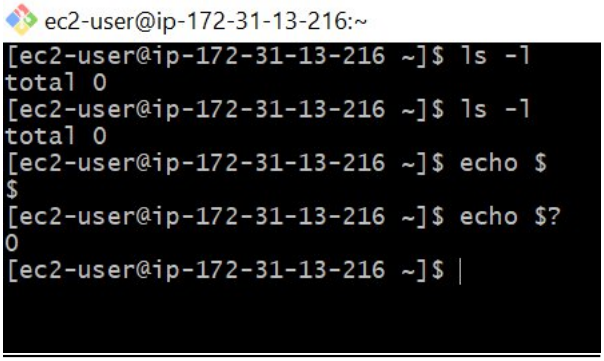
BASH SCRIPTING

1, Tasks to Execute

1 Execute all scripts from the devops-cloud-documents-channel.

Exit status:



ls -l displays files and directories in the current directory.

echo $? displays the exit status of the most recently executed command.

2, Check a successful Bash script

1. `#!/bin/bash` specifies the Bash interpreter.
2. `mkdir testdir` attempts to create a directory named `testdir`.
3. `$?` stores the exit status of the last command; `0` means success.
4. The `if-else` statement displays whether the command succeeded or failed.

```
#!/bin/bash

mkdir testdir

if [ $? -eq 0 ]; then

   echo "Script executed successfully"

else

   echo "Script execution failed"
```

fi

```
[root@ip-172-31-13-216 teja]# vi hello
[root@ip-172-31-13-216 teja]# ./hello

hello

0
[root@ip-172-31-13-216 teja]# cat hello
#!/bin/bash

echo
echo hello
echo
echo $?
[root@ip-172-31-13-216 teja]# chmod 777 hello
[root@ip-172-31-13-216 teja]# ll
total 4
-rwxrwxrwx. 1 root root 43 Oct 27 06:53 hello
[root@ip-172-31-13-216 teja]# ./hello

hello

0
[root@ip-172-31-13-216 teja]#
```

3, Return Un-Successful Exist Status

```
[root@ip-172-31-13-216 teja]# cat hello
#!/bin/bash

echo
echi hello
echo
echo $?
[root@ip-172-31-13-216 teja]# chmod 777 hello
[root@ip-172-31-13-216 teja]# ll
total 4
-rwxrwxrwx. 1 root root 43 Oct 27 06:56 hello
[root@ip-172-31-13-216 teja]# ./hello

./hello: line 4: echi: command not found

0
[root@ip-172-31-13-216 teja]#
```

4, Return exist status if file exist

- `#!/bin/bash` specifies the Bash interpreter.
- `-f "sample.txt"` checks whether the file exists as a regular file.
- `exit 0` indicates success if the file exists.
- `exit 1` indicates failure if the file does not exist.

```
[root@ip-172-31-13-216 teja]# ll .files.bash
-rwxrwxrwx. 1 root root 98 Oct 27 07:14 .files.bash
[root@ip-172-31-13-216 teja]# cat .files.bash
#!/bin/bash

ls -l /ravi/teja/

if [ $? -eq 0 ]
then
echo "file exisit"
else
echo "not exisit"
fi
[root@ip-172-31-13-216 teja]# chmod 777 .files.bash
[root@ip-172-31-13-216 teja]# ./.files.bash
total 4
-rw-r--r--. 1 root root  0 Oct 27 07:14 files
-rw-r--r--. 1 root root  0 Oct 27 07:12 files.bash
-rwxrwxrwx. 1 root root 43 Oct 27 06:56 hello
file exisit
[root@ip-172-31-13-216 teja]#
```

5, Check the day of the week

```
[root@ip-172-31-13-216 teja]# vi checkaday
[root@ip-172-31-13-216 teja]# cat checkaday
#!/bin/bash
a=$(date +%a)
if [ "$a" = "Monday" ]
then
echo "today is $a"
else
echo "today is $a"
fi
[root@ip-172-31-13-216 teja]# ./checkaday
today is Tue
[root@ip-172-31-13-216 teja]#
```

1. `#!/bin/bash` specifies the Bash interpreter.
2. `date +%A` retrieves the current day of the week in full (e.g., Monday).
3. `day=$(...)` stores the day in the `day` variable.
4. `echo` displays the current day on the terminal.

#!/bin/bash

day=$(date +%A)

echo "Today is $day"

Bash Script Tasks

Create a bash script to check if a directory is available or not.

```
[root@ip-172-31-13-216 teja]# vi checkdir
[root@ip-172-31-13-216 teja]# chmod 755 checkdir
[root@ip-172-31-13-216 teja]# cat checkdir
#!/bin/bash

echo " enter  a dir"
read -r dir

if [ -d "$dir" ]
then
echo "directory is exisit"
else
echo "dir is not exisit"
fi
[root@ip-172-31-13-216 teja]# ll checkdir
-rwxr-xr-x. 1 root root 132 Oct 27 07:55 checkdir
[root@ip-172-31-13-216 teja]# bash checkdir
 enter  a dir
ravi
dir is not exisit
[root@ip-172-31-13-216 teja]# teja
teja: Command not found.
[root@ip-172-31-13-216 teja]# root
root: Command not found.
[root@ip-172-31-13-216 teja]# pwd
/ravi/teja
[root@ip-172-31-13-216 teja]# bash checkdir
 enter  a dir
/ravi/teja
directory is exisit
[root@ip-172-31-13-216 teja]#
```

read -r dir reads the directory path entered by the user.

-d "$dir" checks whether the path exists as a directory.

Create a bash script to create multiple files.

1. `#!/bin/bash` specifies that the script runs using Bash.
2. `for i in {1..5}` loops through numbers 1 to 5.
3. `touch "file$i.txt"` creates five files named `file1.txt` to `file5.txt`.
4. `echo` displays a success message after creating the files.

#!/bin/bash

```
for i in {1..5}

do

   touch "file$i.txt

done

echo "5 files created successfully"
```

```
[root@ip-172-31-13-216 teja]# vi createfiles
[root@ip-172-31-13-216 teja]# chmod +x createfiles
[root@ip-172-31-13-216 teja]# bash createfiles
enter to create a file
raju
All files are created
[root@ip-172-31-13-216 teja]# ll
total 16
-rwxrwxrwx. 1 root root 102 Oct 27 07:45 checkaday
-rwxr-xr-x. 1 root root 132 Oct 27 07:55 checkdir
-rwxr-xr-x. 1 root root 157 Oct 27 08:33 createfiles
-rw-r--r--. 1 root root   0 Oct 27 07:14 files
-rw-r--r--. 1 root root   0 Oct 27 07:12 files.bash
-rwxrwxrwx. 1 root root  43 Oct 27 06:56 hello
[root@ip-172-31-13-216 teja]# vi createfiles
[root@ip-172-31-13-216 teja]# ./createfiles
enter to create a file
5
created file1.txt
created file2.txt
created file3.txt
created file4.txt
created file5.txt
All files are created
```

read -r n — reads how many files you want to create.

for ((i=1; i<=n; i++)) — repeats the loop from 1 to n.

touch "file$i.txt" — creates an empty file, or updates the timestamp if that file already exists.

Create a bash script to take a backup of a directory.

```
[root@ip-172-31-13-216 teja]# vi backup.sh
[root@ip-172-31-13-216 teja]# chmod +x backup.sh
[root@ip-172-31-13-216 teja]# ./backup.sh
Enter directory path to back up:
/ravi/teja
Backup completed successfully
Backup file: /tmp/backup_20261027_094901.tar.gz
[root@ip-172-31-13-216 teja]# cat backup.sh
#!/bin/bash

echo "Enter directory path to back up:"
read -r dir

if [ -d "$dir" ]; then
    backup="/tmp/backup_$(date +%Y%m%d_%H%M%S).tar.gz"

    if tar -czf "$backup" -C "$(dirname "$dir")" "$(basename "$dir")"; then
        echo "Backup completed successfully"
        echo "Backup file: $backup"
    else
        echo "Backup failed"
        rm -f "$backup"
    fi
else
    echo "Directory does not exist"
fi
[root@ip-172-31-13-216 teja]#
```

-d "$dir" Checks whether the directory exists

date +%Y%m%d_%H%M%S Generates a timestamp

tar Creates an archive

-c Creates a new archive

-z Compresses using gzip

-f Specifies the archive filename

$? Gets the exit status of the previous command

Create a bash script to install Nginx on an EC2 server.

Steps to Install Nginx on an AWS EC2 Server

Step 1: Connect to your EC2 instance using SSH or Git Bash.

Step 2: Create a Bash script.

```
[root@ip-172-31-13-216 teja]# cat instal_nginx.sh
#!/bin/bash

set -e

echo "Starting Nginx installation..."

if [ -f /etc/os-release ]; then
    . /etc/os-release
else
    echo "Cannot detect operating system."
    exit 1
fi

case "$ID" in
    ubuntu)
        sudo apt-get update
        sudo apt-get install -y nginx
        ;;
    amzn|rhel|centos|rocky|almalinux)
        if command -v dnf >/dev/null 2>&1; then
            sudo dnf install -y nginx
        else
            sudo yum install -y nginx
        fi
        ;;
    *)
        echo "Unsupported OS: $ID"
        exit 1
        ;;
esac

sudo systemctl enable --now nginx

echo "Checking Nginx status..."
sudo systemctl status nginx --no-pager

echo "Nginx installation completed successfully!"
[root@ip-172-31-13-216 teja]# bash instal_nginx.sh
Starting Nginx installation...
Last metadata expiration check: 1 day, 0:55:39 ago on Mon Oct 26 09:11:26 2026.
Package nginx-1:1.30.5-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
Checking Nginx status...
● nginx.service - The nginx HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
     Active: active (running) since Mon 2026-10-26 10:19:59 UTC; 23h ago
```

Give execute permission.

```
chmod +x install_nginx.sh
```

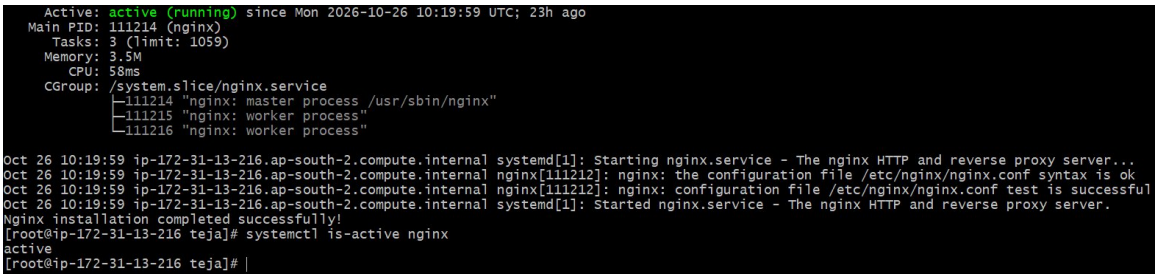
Step 6: Run the script.

```
./install_nginx.sh
```

Step 7: Verify Nginx is running.

```
curl http://localhost
```

Step 8: Access Nginx in your browser using `http://YOUR-EC2-PUBLIC-IP`. Ensure your EC2 security group allows inbound HTTP traffic on port 80.





Create a bash script to install Apache Tomcat on an EC2 server.

Bash Script to Install Apache Tomcat on an EC2 Server

Step 1: Connect to your EC2 instance

Connect to your Ubuntu EC2 instance using SSH or Git Bash.

Step 2: Create the Bash script

```
vi install_tomcat.sh
```

Step 3: Add the following script

```
[root@ip-172-31-13-216 teja]# vi install_tomcat.sh
[root@ip-172-31-13-216 teja]# chmod 777 install_tomcat.sh
[root@ip-172-31-13-216 teja]# cat install_tomcat.sh
#!/bin/bash

set -e

echo "Starting Apache Tomcat installation..."

# Detect operating system
. /etc/os-release

# Install Java
case "$ID" in
    ubuntu)
        apt-get update
        apt-get install -y openjdk-17-jdk curl tar
        ;;
    amzn|rhel|centos|rocky|almalinux)
        if command -v dnf >/dev/null 2>&1; then
            dnf install -y java-17-amazon-corretto-devel curl tar
        else
            yum install -y java-17-amazon-corretto-devel curl tar
        fi
        ;;
    *)
        echo "Unsupported operating system: $ID"
        exit 1
        ;;
esac

# Create a dedicated Tomcat user
if ! id tomcat >/dev/null 2>&1; then
    useradd --system --home-dir /opt/tomcat \
        --shell /sbin/nologin tomcat
fi

# Download and install Tomcat
TOMCAT_VERSION="9.0.115"
INSTALL_DIR="/opt/tomcat"

if [ ! -f "$INSTALL_DIR/bin/catalina.sh" ]; then
    mkdir -p "$INSTALL_DIR"
    curl -fL \
      "https://archive.apache.org/dist/tomcat/tomcat-9/v${TOMCAT_VERSION}/bin/apache-tomcat-${TOMCAT_VERSION}.tar.gz" \
      -o /tmp/tomcat.tar.gz

    tar -xzf /tmp/tomcat.tar.gz \
        --strip-components=1 -C "$INSTALL_DIR"
    rm -f /tmp/tomcat.tar.gz
fi

chown -R tomcat:tomcat "$INSTALL_DIR"
chmod +x "$INSTALL_DIR/bin/"*.sh

# Configure Tomcat to listen on port 8082
sed -i 's/port="8080"/port="8082"/' \
    "$INSTALL_DIR/conf/server.xml"

# Create systemd service
cat > /etc/systemd/system/tomcat.service <<'EOF'
[Unit]
Description=Apache Tomcat
After=network.target

[Service]
Type=forking
User=tomcat
Group=tomcat
Environment="JAVA_HOME=/usr/lib/jvm"
Environment="CATALINA_HOME=/opt/tomcat"
Environment="CATALINA_BASE=/opt/tomcat"
Environment="CATALINA_PID=/opt/tomcat/temp/tomcat.pid"
ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh
Restart=on-failure

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now tomcat

echo "Tomcat installation finished."
[root@ip-172-31-13-216 teja]# ./install_tomcat.sh
Starting Apache Tomcat installation...
Last metadata expiration check: 1 day, 1:05:49 ago on Mon Oct 26 09:11:26 2026.
Package java-17-amazon-corretto-devel-1:17.0.20+13-1.amzn2023.1.x86_64 is already installed.
Package tar-2:1.34-1.amzn2023.0.4.x86_64 is already installed.
Error:
 Problem: problem with installed package curl-minimal-8.21.0-5.amzn2023.0.2.x86_64
  - package curl-minimal-8.21.0-5.amzn2023.0.2.x86_64 from @System conflicts with curl provided by curl-7.87.0-2.amzn2023.0.2.x86_64 from amazonlinux
  - package curl-minimal-7.87.0-2.amzn2023.0.2.x86_64 from amazonlinux conflicts with curl provided by curl-7.87.0-2.amzn2023.0.2.x86_64 from amazonlinux
  - package curl-minimal-7.88.0-1.amzn2023.0.1.x86_64 from amazonlinux conflicts with curl provided by curl-7.87.0-2.amzn2023.0.2.x86_64 from amazonlinux
  - package curl-minimal-7.88.1-1.amzn2023.0.1.x86_64 from amazonlinux conflicts with curl provided by curl-7.87.0-2.amzn2023.0.2.x86_64 from amazonlinux
  - package curl-minimal-8.0.1-1.amzn2023.0.1.x86_64 from amazonlinux conflicts with curl provided by curl-7.87.0-2.amzn2023.0.2.x86_64 from amazonlinux
  - package curl-minimal-8.0.1-1.amzn2023.x86_64 from amazonlinux conflicts with curl provided by curl-7.87.0-2.amzn2023.0.2.x86_64 from amazonlinux
```

Save and exit

Press `Esc`, type `:wq`, and press `Enter`.

Step 5: Give execute permission

```
chmod +x install_tomcat.sh
```

Step 6: Run the script

```
./install_tomcat.sh
```

Step 7: Verify Tomcat

```
curl http://localhost:8080
```

If Tomcat is working, you should receive its default web page HTML.

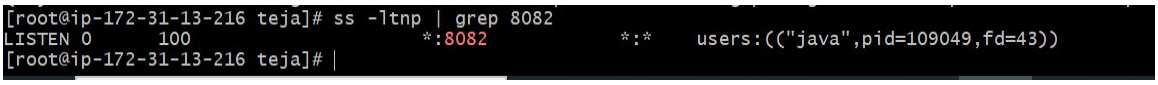
Step 8: Access Tomcat in your browser

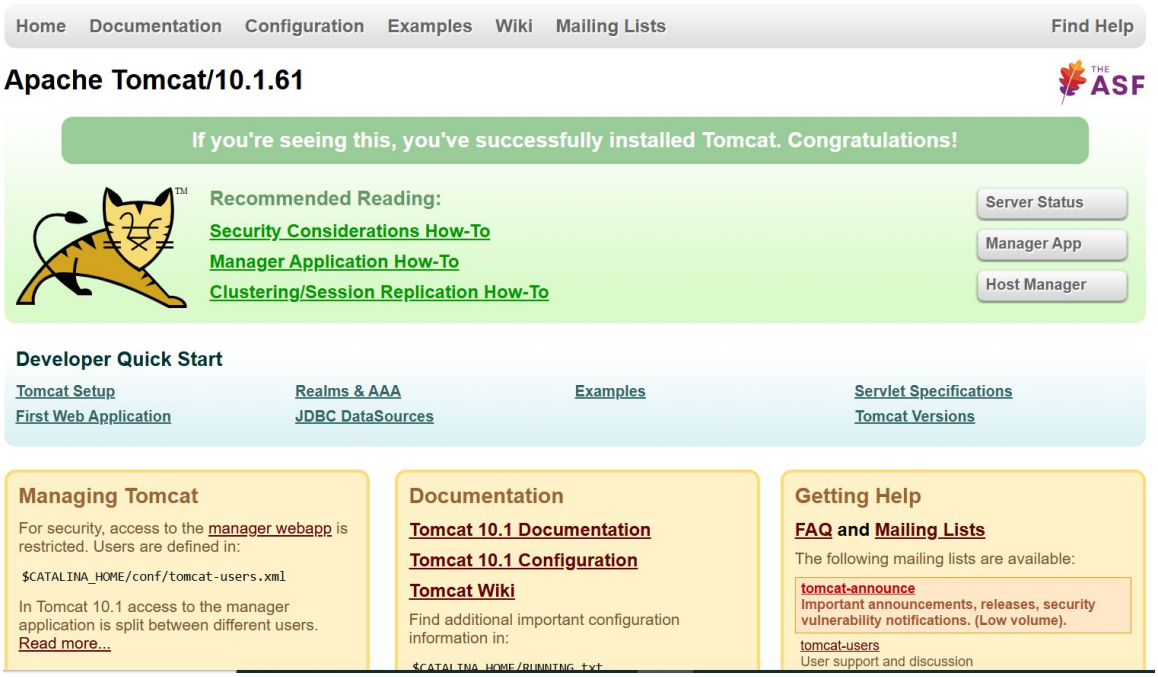
Open:

```
http://YOUR-EC2-PUBLIC-IP:8080
```

In the EC2 security group, allow inbound TCP traffic on port 8080 from your IP address.

Note: This script is for Ubuntu EC2 instances. The `JAVA_HOME` path is Ubuntu-specific; Amazon Linux requires a different Java path and package installation commands.





Create a bash script to check if the Nginx service is running, if not running then script should start the service.

1. `#!/bin/bash` specifies the Bash interpreter.
2. `systemctl is-active --quiet nginx` checks whether the Nginx service is running.
3. If Nginx is running, the script displays a message; otherwise, it starts Nginx.
4. `sudo systemctl start nginx` starts the service if it is stopped.

```
#!/bin/bash

if systemctl is-active --quiet nginx; then

    echo "Nginx is already running"

else

    sudo systemctl start nginx
```

```
    echo "Nginx service started"
fi
```

```
[root@ip-172-31-13-216 teja]# vi check_nginx.sh
[root@ip-172-31-13-216 teja]# chmod 755 check_nginx.sh
[root@ip-172-31-13-216 teja]# cat check_nginx.sh
#!/bin/bash

if systemctl is-active --quiet nginx
then
    echo "Nginx is already running"
else
    echo "Nginx is not running. Starting Nginx..."

    if systemctl start nginx
    then
        echo "Nginx started successfully"
    else
        echo "Failed to start Nginx"
        exit 1
    fi
fi
[root@ip-172-31-13-216 teja]# bash check_nginx.sh
Nginx is already running
[root@ip-172-31-13-216 teja]#
```

Create a bash script for a calculator.

The objective of this script is to create a simple calculator using Bash scripting in Linux. It performs basic arithmetic operations such as addition, subtraction, multiplication, and division based on user input.

1. `#!/bin/bash`

This is called the shebang. It specifies that the script should be executed using the Bash shell.

2. `echo`

The `echo` command displays messages on the terminal, asking the user to enter numbers and an arithmetic operator.

3. `read -r a`

The `read` command accepts user input and stores the first number in the variable `a`. The `-r` option prevents backslashes from being interpreted as escape characters.

4. `read -r op`

This command accepts the arithmetic operator (`+`, `-`, `*`, or `/`) and stores it in the variable `op`.

5. `read -r b`

This command accepts the second number and stores it in the variable `b`.

6. `case "$op" in`

The `case` statement checks which operator the user entered and executes the corresponding operation.

- `+)` performs addition.
- `-)` performs subtraction.
- `\*)` performs multiplication. The asterisk is escaped because it has a special meaning in shell patterns.
- `/)` performs division.

7. Arithmetic expansion: `$(( ))`

Bash arithmetic expansion calculates the result of an arithmetic expression.

For example:

```
result=$((a + b))
```

If `a=20` and `b=10`, the result is `30`.

8. Division-by-zero validation

```
if [ "$b" -eq 0 ]; then
```

This condition checks whether the second number is zero before performing division. Division by zero is not allowed, so the script displays an error and exits.

9. Default case: `*)`

If the user enters an unsupported operator, the script displays `Invalid operator` and exits with a nonzero status.

10. `esac`

The `esac` keyword marks the end of the `case` statement.

11. `echo "Result: $result"`

This command displays the calculated result on the terminal.

```
[root@ip-172-31-13-216 teja]# vi calculator
[root@ip-172-31-13-216 teja]# chmod +x calculator
[root@ip-172-31-13-216 teja]# cat calculator
#!/bin/bash

echo "===== Bash Calculator ====="

echo "Enter first number:"
read -r a

echo "Enter second number:"
read -r b

echo "Choose an operation:"
echo "1. Addition (+)"
echo "2. Subtraction (-)"
echo "3. Multiplication (*)"
echo "4. Division (/)"

read -r choice

case "$choice" in
    1)
        result=$(echo "$a + $b" | bc)
        echo "Result: $result"
        ;;
    2)
        result=$(echo "$a - $b" | bc)
        echo "Result: $result"
        ;;
    3)
        result=$(echo "$a * $b" | bc)
        echo "Result: $result"
        ;;
    4)
        if [ "$b" = "0" ]; then
            echo "Error: Division by zero is not allowed"
        else
            result=$(echo "scale=2; $a / $b" | bc)
            echo "Result: $result"
        fi
        ;;
    *)
        echo "Invalid choice"
        ;;
esac
```

```
[root@ip-172-31-13-216 teja]# ./calculator
===== Bash Calculator =====
Enter first number:
28
Enter second number:
9
Choose an operation:
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
/
Invalid choice
[root@ip-172-31-13-216 teja]# ./calculator
===== Bash Calculator =====
Enter first number:
90000
Enter second number:
90000
Choose an operation:
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
3
Result: 8100000000
[root@ip-172-31-13-216 teja]#
```

Create a bash script to check if a directory exists, if not then create a directory.

```
[root@ip-172-31-13-216 teja]# vi checkdir.sh
[root@ip-172-31-13-216 teja]# chmod 777 checkdir.sh
[root@ip-172-31-13-216 teja]# cat checkdir.sh
#!/bin/bash

echo "Enter directory name:"
read -r dir

if [ -d "$dir" ]
then
    echo "Directory already exists"
else
    mkdir -p "$dir"

    if [ $? -eq 0 ]
    then
        echo "Directory created successfully"
    else
        echo "Failed to create directory"
        exit 1
    fi
fi
[root@ip-172-31-13-216 teja]# ./checkdir.sh
Enter directory name:
rajendhar
Directory created successfully
```

read -r dir — reads the directory name from the user.

[-d "$dir"] — checks whether the directory exists.

mkdir -p "$dir" — creates the directory, including missing parent directories.

$? — checks the exit status of the previous command.

exit 1 — indicates failure.

Create a bash script to delete the last 3 lines of a file.

```
[root@ip-172-31-13-216 teja]# vi ART
[root@ip-172-31-13-216 teja]# vi delete_last3.sh
[root@ip-172-31-13-216 teja]# cat delete_last3.sh

#!/bin/bash

echo "Enter file name:"
read -r file

if [ -f "$file" ]; then
    sed -i '$d' "$file"
    sed -i '$d' "$file"
    sed -i '$d' "$file"
    echo "Last 3 lines deleted successfully"
else
    echo "File does not exist"
    exit 1
fi

[root@ip-172-31-13-216 teja]# ch delete_last3.sh
chacl          chattr         chcpu          chec
chage          chcat          chdir          chec
chardetect     chcon          check_nginx.sh chec
[root@ip-172-31-13-216 teja]# chmod 777 delete_la
[root@ip-172-31-13-216 teja]# bash delete_last3.s
Enter file name:
ART
Last 3 lines deleted successfully
[root@ip-172-31-13-216 teja]# cat ART
My self raviteja
i have completed my grad in snist
iam from hayathnagr
i have completed my crouse in devops engineering
[root@ip-172-31-13-216 teja]#
```

-f "$file" checks whether the file exists.

sed -i '$d' "$file" deletes the last line of the file in place.

Repeating the command three times deletes the last three lines.

Bash script to monitor cpu and if it is more than 80% then send email notification.

```
[root@ip-172-31-13-216 teja]# vi cpu_monitor.sh
[root@ip-172-31-13-216 teja]# cat cpu_monitor.sh

#!/bin/bash

THRESHOLD=80
EMAIL="asgoud1515@gmail.com"

CPU_IDLE=$(top -bn2 -d 1 | grep "Cpu(s)" | tail -1 | sed 's/.*, *\([0-9.]*\)%* id.*/\1/')

CPU_USAGE=$(echo "$CPU_IDLE" | sed 's/.*/100-&/' | bc)

echo "Current CPU usage: $CPU_USAGE%"

if (( $(echo "$CPU_USAGE > $THRESHOLD" | bc -l) )); then
    echo "Warning: CPU usage is above ${THRESHOLD}%!" |
        mail -s "CPU Usage Alert" "$EMAIL"
    echo "Email alert triggered."
else
    echo "CPU usage is normal."
fi


[root@ip-172-31-13-216 teja]# chmod 755 cpu_monitor.sh
[root@ip-172-31-13-216 teja]# bash cpu_monitor.sh
Current CPU usage: .5%
CPU usage is normal.
[root@ip-172-31-13-216 teja]#
```

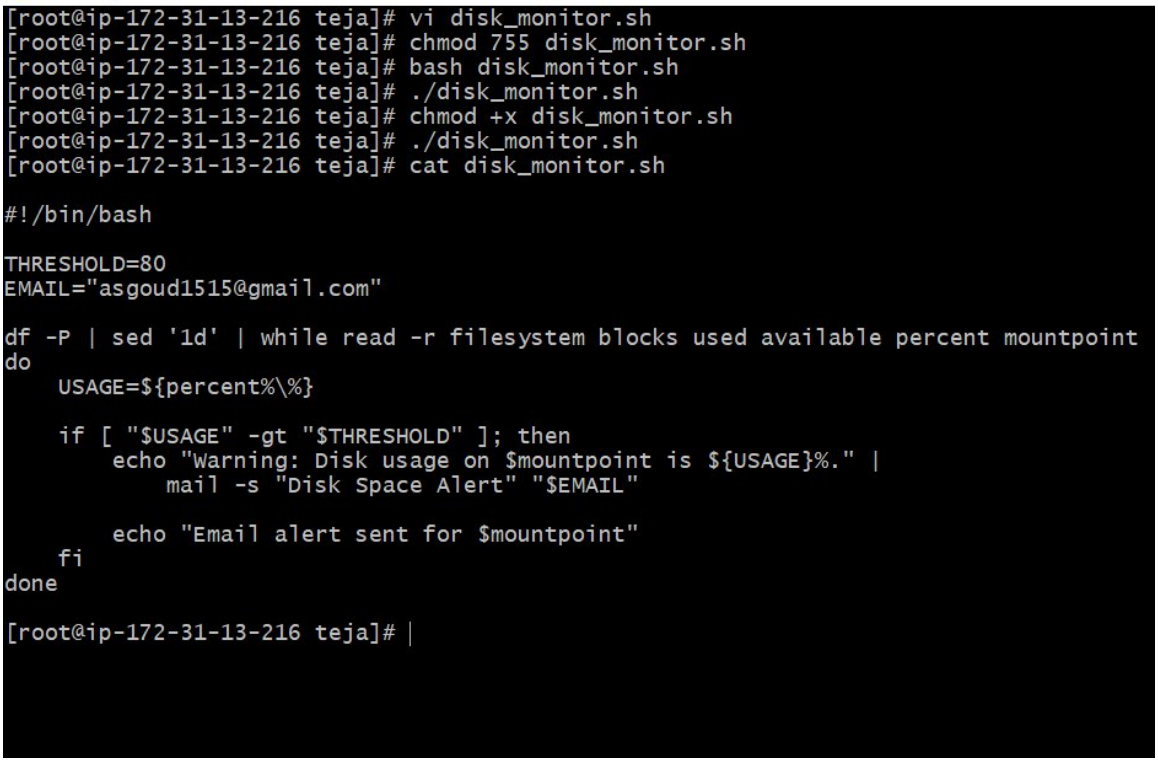
1. sed extracts the idle percentage from the top command output.

2. bc calculates CPU usage: 100 - idle percentage.

3. bc -l compares CPU usage with the threshold of 80%.

4. mail sends the alert if CPU usage exceeds 80%

Bash script to monitor disk space and if it is more than 80% then send email notification.



```
[root@ip-172-31-13-216 teja]# vi disk_monitor.sh
[root@ip-172-31-13-216 teja]# chmod 755 disk_monitor.sh
[root@ip-172-31-13-216 teja]# bash disk_monitor.sh
[root@ip-172-31-13-216 teja]# ./disk_monitor.sh
[root@ip-172-31-13-216 teja]# chmod +x disk_monitor.sh
[root@ip-172-31-13-216 teja]# ./disk_monitor.sh
[root@ip-172-31-13-216 teja]# cat disk_monitor.sh

#!/bin/bash

THRESHOLD=80
EMAIL="asgoud1515@gmail.com"

df -P | sed '1d' | while read -r filesystem blocks used available percent mountpoint
do
    USAGE=${percent%\%}

    if [ "$USAGE" -gt "$THRESHOLD" ]; then
        echo "Warning: Disk usage on $mountpoint is ${USAGE}%." |
            mail -s "Disk Space Alert" "$EMAIL"

        echo "Email alert sent for $mountpoint"
    fi
done

[root@ip-172-31-13-216 teja]#
```

df -P displays disk usage in a consistent format.

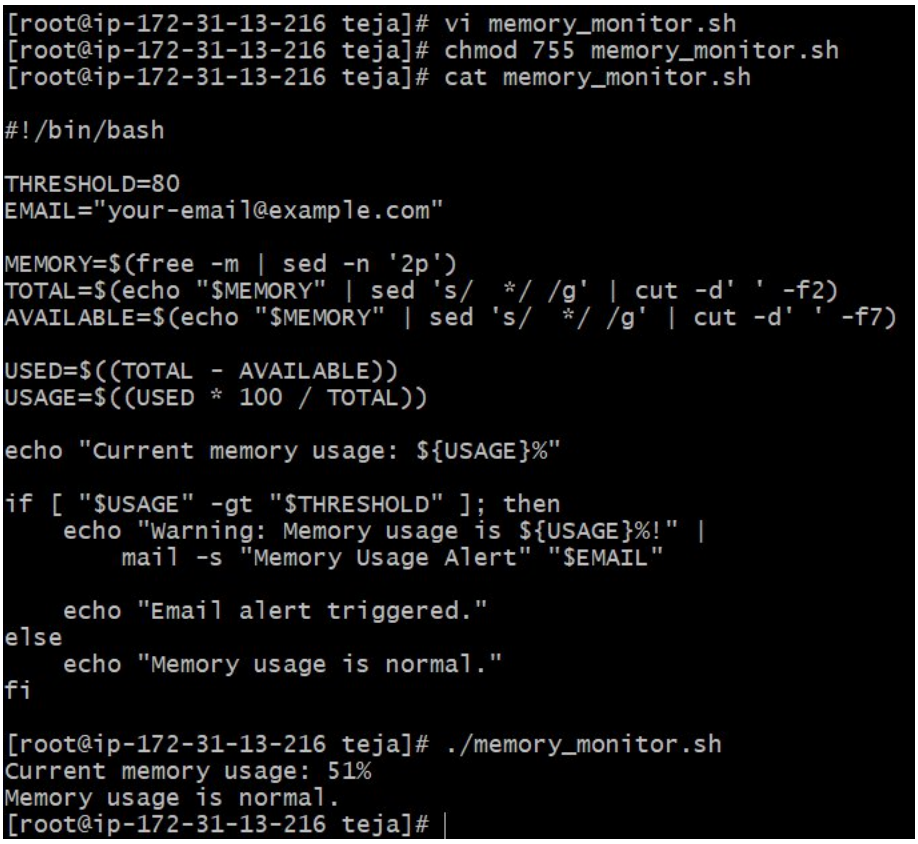
sed '1d' removes the header row.

${percent%\%} removes the % symbol.

-gt 80 checks whether usage is greater than 80%.

mail sends an email alert

Bash script to monitor memory and if it is more than 80% then send email notification.



```
[root@ip-172-31-13-216 teja]# vi memory_monitor.sh
[root@ip-172-31-13-216 teja]# chmod 755 memory_monitor.sh
[root@ip-172-31-13-216 teja]# cat memory_monitor.sh

#!/bin/bash

THRESHOLD=80
EMAIL="your-email@example.com"

MEMORY=$(free -m | sed -n '2p')
TOTAL=$(echo "$MEMORY" | sed 's/  */ /g' | cut -d' ' -f2)
AVAILABLE=$(echo "$MEMORY" | sed 's/  */ /g' | cut -d' ' -f7)

USED=$((TOTAL - AVAILABLE))
USAGE=$((USED * 100 / TOTAL))

echo "Current memory usage: ${USAGE}%"

if [ "$USAGE" -gt "$THRESHOLD" ]; then
    echo "Warning: Memory usage is ${USAGE}%!" |
        mail -s "Memory Usage Alert" "$EMAIL"

    echo "Email alert triggered."
else
    echo "Memory usage is normal."
fi

[root@ip-172-31-13-216 teja]# ./memory_monitor.sh
Current memory usage: 51%
Memory usage is normal.
[root@ip-172-31-13-216 teja]#
```

free -m displays memory in megabytes.

sed extracts the memory information.

cut retrieves total and available memory values.

Bash calculates the percentage used.

if ["$USAGE" -gt 80] checks whether usage exceeds 80%.

mail sends an alert when the threshold is exceeded.

Exercise: Crontab Entries

1, April 5th Midnight.

Open the crontab editor

```
[root@ip-172-31-2-45 ec2-user]# crontab -e
```

Enter the complete line

```
crontab: installing new crontab
[root@ip-172-31-2-45 ec2-user]# crontab -l
0 0 5 4 * echo "April 5 midnight" >> /tmp/cron-test.log
[root@ip-172-31-2-45 ec2-user]#
```

This means:

- `0` — Minute 0
- `0` — Hour 0 (midnight)
- `5` — Day of the month
- `4` — April
- `*` — Every day of the week
- `echo ...` — The command that runs at the scheduled time

- **Verify the entry**
- `crontab -l`

Crontab has two parts

```
0 0 5 4 *  command_to_execute
```

- Left side: When should the task run?
- Right side: What should the computer do at that time?

2, 5th of Every November, January, June if it is a Thursday

For the 5th of November, January, and June, only if it is a Thursday, the crontab expression is:

0 0 5 11,1,6 4

```
[root@ip-172-31-2-45 ec2-user]# crontab -e
```

root@ip-172-31-2-45:/home/ec2-user

```
0 0 5 11,1,6 * [ "$(date +\%u)" -eq 4 ] && echo "It is Thursday" >> /tmp/thursday.log
~
```

Explanation

Field	Value	Meaning
Minute	`5,27`	At minutes 5 and 27
Hour	`9,10,11`	At 9 AM, 10 AM, and 11 AM
Day of month	`*`	Every day
Month	`*`	Every month
Day of week	`*`	Every day of the week

```
crontab: installing new crontab
[root@ip-172-31-2-45 ec2-user]# crontab -l
0 0 5 11,1,6 * [ "$(date +\%u)" -eq 4 ] && echo "It is Thursday" >> /tmp/thursday.log
[root@ip-172-31-2-45 ec2-user]#
```

3, At 05 and 27th minutes of 9, 10, 11 hours every day

For 05 and 27th minutes of the 9, 10, and 11 hours every day, the crontab expression is:

```
5,27 9,10,11 * * *
```

Field	Value	Meaning
Minute	`5,27`	At minutes 5 and 27
Hour	`9,10,11`	At 9 AM, 10 AM, and 11 AM
Day of month	`*`	Every day
Month	`*`	Every month
Day of week	`*`	Every day of the week

root@ip-172-31-2-45:/home/ec2-user

```
5,27 9,10,11 * * * echo "Cron job executed" >> /tmp/cron-test.log
~
```

```
[root@ip-172-31-2-45 ec2-user]# crontab -l
5,27 9,10,11 * * * echo "Cron job executed" >> /tmp/cron-test.log
[root@ip-172-31-2-45 ec2-user]#
```

4, 34 minutes of 9th hour on 15th August

root@ip-172-31-2-45:/home/ec2-user

```
34 9 15 8 * echo "Cron job executed" >> /tmp/cron-test.log
~
~
```

Explanation

Field	Value	Meaning
Minute	`34`	At minute 34
Hour	`9`	At 9 AM
Day of month	`15`	On the 15th
Month	`8`	August
Day of week	`*`	Every day of the week

```
[root@ip-172-31-2-45 ec2-user]# crontab -e
crontab: installing new crontab
[root@ip-172-31-2-45 ec2-user]# crontab -l
34 9 15 8 * echo "Cron job executed" >> /tmp/cron-test.log
[root@ip-172-31-2-45 ec2-user]#
```

5, Every midnight

Explanation

Field	Value	Meaning
Minute	`0`	At minute zero
Hour	`0`	At midnight (12:00 AM)
Day of month	`*`	Every day
Month	`*`	Every month
Day of week	`*`	Every day of the week

```
[root@ip-172-31-2-45 ec2-user]# crontab -e
crontab: installing new crontab
[root@ip-172-31-2-45 ec2-user]# crontab -l
0 0 * * * echo "Cron job executed" >> /tmp/cron-test.log
[root@ip-172-31-2-45 ec2-user]#
```

6, Every Weekend (Saturday night 11:59 PM)

root@ip-172-31-2-45:/home/ec2-user

```
59 23 * * 6 echo "Cron job executed" >> /tmp/cron-test.log
~
```

Explanation

Field	Value	Meaning
Minute	`59`	At minute 59
Hour	`23`	At 11 PM
Day of month	`*`	Every day of the month
Month	`*`	Every month
Day of week	`6`	Saturday

root@ip-172-31-2-45:/home/ec2-user

```
0 0 * * * echo "Cron job executed" >> /tmp/cron-test.log
~
[root@ip-172-31-2-45 ec2-user]# crontab -e
crontab: installing new crontab
[root@ip-172-31-2-45 ec2-user]# crontab -l
59 23 * * 6 echo "Cron job executed" >> /tmp/cron-test.log
[root@ip-172-31-2-45 ec2-user]#
```

7, After every reboot

To run a cron job after every reboot, use the `@reboot` keyword.

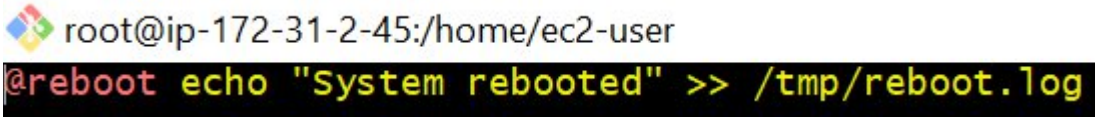
Syntax:

```
@reboot command
```

Example: Run a script after every reboot.

1. Open the crontab editor:

   ```
   crontab -e
   ```

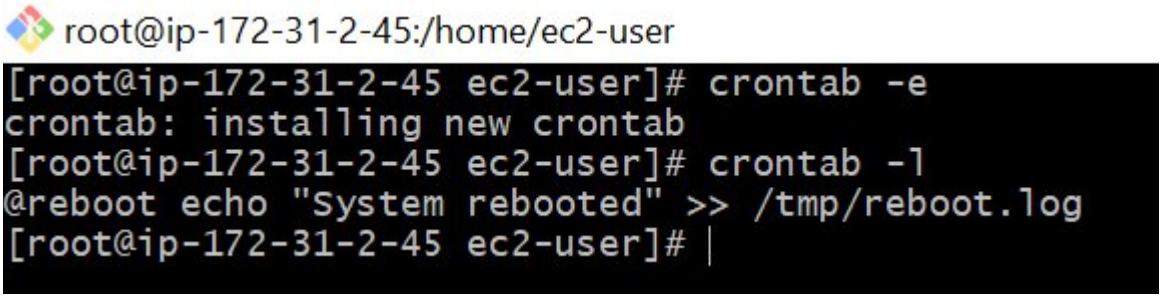
   

2. Add this line:

   ```
   @reboot echo "System rebooted" >> /tmp/reboot.log
   ```

3. Save and exit.
4. After the next reboot, check the log:

   ```
   cat /tmp/reboot.log
   ```

Explanation:

Part	Meaning
`@reboot`	Runs the command when cron starts after system startup
`echo`	Prints a message
`>>`	Appends the message to a file
`/tmp/reboot.log`	Stores the output

Note: The cron service must be installed and running. `@reboot` runs when the cron daemon starts, not necessarily after every service or network connection is ready.